

Supplementary Material: Deciding If, When, and Where to Replan for Robust Manipulation

Anonymous Author(s)

Submission Id: 2220

S1 Overview

This supplement provides technical detail that complements the main paper. Section S2 gives the goal instructions, the evaluation conditions, the hidden contents of each variant and the trial randomization. Section S3 describes perception and memory, the *IF*, *WHEN* and *WHERE* decisions, validation, recovery and termination, and the simulation controllers. Section S4 shows the planner prompts, the in-context examples and a recorded discovery correction with validation feedback. Section S5 lists the adaptations of the external baselines that affect the comparison, and Section S6 the inference settings and trial protocol. Notation follows the main paper.

S2 Task Details

The simulation evaluation uses 14 MuJoCo variants: nine in the kitchen domain and five in the grill domain. Every trial of a domain receives the same natural-language goal g :

- Kitchen: “move ALL THE GROCERIES inside the cupboard and ALL THE MUGS inside the box”.
- Grill: “COOK all raw meat using the grill and SERVE all cooked meat on the PLATE in the serving area.”

Goal evaluation. The required relations $\mathcal{R}_\tau$ place each mug in the box, each grocery on the cupboard shelf, the plate in the serving area and each meat on the plate; a phone that overlaps a placement area must end outside every placement area. In the grill, $\mathcal{P}_\tau$ requires every meat to be cooked, not cooked again once cooked, and not served raw. Cooking is determined from h_T : a meat completes one cooking cycle when it is inside the grill as the lid closes and is still inside when the lid next opens; meat discovered already cooked starts with one completed cycle. Ordering constraints (obstructing objects leave a placement area before the next placement into it; cooked meat leaves the grill before the next close; raw meat completes a cycle before plating) are reported as insertion violations and do not change SR or PGC.

Placement areas. Overlap is evaluated against fixed areas that the executor uses for placement: the robot-side half of the box interior and the grill slot nearest the robot, where every meat is placed. Hidden objects that are meant not to obstruct lie in the far half of the box or in the far grill slots. The areas are used only by the system and never appear in a prompt.

Trial randomization. Seeds 0–9 perturb the initial pose of each variant’s free objects by up to ± 3 cm in x and y and $\pm 20^\circ$ in yaw. A sample is rejected if the object touches another movable object or its height changes by more than 1 cm after settling; after 30 rejected

Table S1: Hidden contents and reference decisions. Ovl.: the hidden object overlaps the placement area of a pending placement. The reference decisions are used only for evaluation and are never given to the planner. Grill urgency follows the cooking procedure rather than overlap.

ID	Hidden objects (location)	Ovl.	Reference decision
<i>Kitchen</i>			
K0	none	–	no correction
K1	phone (box)	✓	correct; urgent
K2	phone (box)	×	no correction
K3	1 grocery (box)	✓	correct; urgent
K3-n2	2 groceries (box)	✓	correct; urgent
K3-n3	3 groceries (box)	✓	correct; urgent
K4	1 grocery (box)	×	correct; deferred
K1-w1	phone (box); +1 grocery on the table	✓	correct; urgent
K1-w2	phone (box); +2 groceries on the table	✓	correct; urgent
<i>Grill</i>			
G0	none	–	no correction
G1	2 cooked meats (grill)	×	correct; urgent
G1-n1	1 cooked meat (grill)	×	correct; urgent
G2	1 cooked + 1 raw meat (grill)	×	correct; cooked: urgent, raw: deferred
G3	2 raw meats (grill)	×	correct; deferred

samples the object keeps its nominal pose. Objects whose pose the executor or the variant depends on stay fixed: the plate, the lids and containers, the mug stored in the cupboard, objects inside the cupboard, and the hidden contents of the box and the grill.

S3 Implementation Details

S3.1 Perception and Scene Memory

Observation in simulation. After every execution bundle and before every FM request, the system renders segmentation masks from the scene cameras. An object is counted in a view if its mask there covers at least 10 pixels (at least 50 in the grill scene), and V_t contains the objects counted in at least one view. The remaining facts come from the simulator: the region of each visible object is assigned by testing its pose against the region bounding boxes, lid states are read from the lid joints, and placement-area overlap is computed from the object’s footprint when it was last visible. Gripper state is maintained by the executor.

Identities and memory. K_t records *which* objects have been observed, and m_t stores *what* was last observed about each of them: its region $r_t(o)$ and the step t of the observation. After each observation,

$$m_t(o) = \begin{cases} (r_t(o), t) & o \in V_t, \\ m_{t-1}(o) & o \in K_{t-1} \setminus V_t, \end{cases} \quad (S1)$$

so an occluded or removed-from-view object keeps its last region and observation time, and no entry is ever deleted. The prompt lists



This work is licensed under a Creative Commons Attribution International 4.0 License.

such objects separately from the visible ones, with their last region and the number of steps since they were seen. An object $o \notin K_t$ has no entry, never appears in a prompt, and a plan that names it is rejected with the same message as for a non-existent name, so the planner cannot distinguish a hidden object from an invalid one. A remembered object can be picked only if its last region is not closed off by a closed lid. Without memory (–Memory), only V_t is available.

S3.2 Repanning and Execution

IF. After each bundle, every $o \in K_t$ other than the lids and the held object is assessed with its current region if visible and its remembered region otherwise. Let C_d be the task-relevant categories of the domain, $cat(o)$ the category of o (mugs and groceries; raw and cooked meat), $r^*(o)$ the goal region of o 's category and $\mathcal{A}(\pi_t)$ the placement areas of the regions that some pending place in π_t targets. The trigger set is

$$O_e = \{o \mid \neg cov(o) \wedge [(cat(o) \in C_d \wedge \neg done(o)) \vee (cat(o) \notin C_d \wedge fp(o) \cap \mathcal{A}(\pi_t) \neq \emptyset)]\}, \quad (S2)$$

where $cov(o)$ holds if a pending pick or place names o , or if the FM answered NO_ACTIONS for o and its recorded region has not changed since; $done(o)$ holds if o 's region is $r^*(o)$ and, for meat, h_t shows it cooked; and $fp(o)$ is its footprint. Coverage checks only that a pending action names the object, not its destination or position. Obstruction is tested against every pending placement regardless of its position in π_t , so the plate, which is not a goal category, triggers only when it obstructs. Overcooking and serving raw are left to the evaluator and do not trigger. If $O_e = \emptyset$, execution continues without an FM call.

WHEN. A trigger request is sent to the FM while the robot executes the bundles of π_t that cannot interfere with the correction. The *affected set* contains the objects of O_e , their goal regions, the staging region S (the area where a correction may set objects aside), the container of a trigger object whose placement area it overlaps, and the lids that close off an affected region or a trigger object's container (closing it would change the object's state, e.g. cook a cooked meat again). For bundle b , let $obj(b)$ be its objects, $src(b)$ the region its pick takes from (projected along the plan, so a second pick of the same object uses the region the first placement put it in) and $dst(b)$ its placement region or, for a lid action, the regions the lid closes off. Bundles are scanned in plan order with blocked sets initialized to $B_o = \text{affected objects}$, $B_d = \text{affected regions}$ and $B_s = B_d \setminus S$:

$$b \text{ eligible} \iff obj(b) \cap B_o = \emptyset \wedge dst(b) \cap B_d = \emptyset \wedge src(b) \cap B_s = \emptyset. \quad (S3)$$

A withheld bundle propagates its dependencies to later bundles: $B_o \leftarrow B_o \cup obj(b)$, $B_d \leftarrow B_d \cup dst(b) \cup src(b)$ and $B_s \leftarrow B_s \cup dst(b)$. Later bundles that use its objects, place into or take from its destination, or place into the region it would vacate are therefore also withheld. Because $S \in B_d$ but $S \notin B_s$, placements into the staging region wait, while removing an object from it remains allowed. Eligible bundles run in their original relative order and may precede an earlier withheld bundle. The FM request lists them as scheduled and excludes them from the remaining plan shown, and the scene description is not advanced by their effects. Execution checks for

the response only between bundles; if no eligible bundle is left, the robot waits. Trigger checks are suspended during the wait and resume after the merge. Without concurrency (–WHEN), execution stops during inference.

WHERE. A trigger request returns corrective blocks for the objects of O_e , not a new task plan. Each block lists the objects it handles, an urgency, an insertion position (front, after a named pending action, or end), a one-sentence reason and its actions; a block with NO_ACTIONS states that its objects need none. Let $\pi_{t'}$ be the remaining plan after the actions completed during inference are removed. Only actions of $\pi_{t'}$ that were not scheduled for concurrent execution are eligible anchors; an anchor on a scheduled action is rejected even if that action has not yet run. With urgent blocks $U_1, \dots, U_k$ in the order returned and deferred blocks D_j placed directly after their anchor or appended,

$$\hat{\pi} = U_1 \cdots U_k \cdot ins(\pi_{t'}, \{D_j\}), \quad \pi_{t'} \text{ is a subsequence of } \hat{\pi}, \quad (S4)$$

so corrective actions are added while the pending actions keep their relative order and identifiers. The planner's urgency and position are used as returned; urgent blocks must be placed at the front.

Validation and recovery. A correction is rejected if its blocks are malformed, handle an object outside O_e , leave an object of O_e unhandled, act on objects other than those they list, contain an incomplete pick–place sequence, or anchor on an ineligible action; if the merged plan fails the plan check (unknown or never-observed names, inconsistent pick–place order, an inaccessible remembered object); or if it places into a region before an overlapping trigger object has been removed from that region's placement area (insertion too late, using the overlap recorded at the trigger). No check tests whether a correction is inserted too early or whether its urgency respects a procedure such as cooking; those remain FM decisions. A rejected correction is re-queried as the same corrective request with the failure code and the offending element added, and the remaining plan is kept. An execution failure (a failed grasp, placement or motion plan, or a violated precondition such as a closed lid) instead requests a full replacement of the remaining plan from the updated scene and history; a failure of a concurrently executed bundle discards the pending correction and is handled the same way. There are no task-level local retries. An inconsistent executor state, such as placing an object that is not held, ends the trial.

Termination. At most $N = 11$ FM calls are made per trial (the initial request and ten replanning calls, re-queries included). A trial also ends when the plan is exhausted with no pending trigger, when a full-plan request returns NO_ACTIONS, on a terminal failure, or when the planner repeats an earlier answer for the same state and failure a second time (the first repetition is re-queried with a note). Termination is not success: success is evaluated afterwards from the final state and execution history with G_{τ} , and no runtime goal check was used in the reported runs.

S3.3 Simulation Controllers

A Franka Panda executes the primitives in MuJoCo. Inverse kinematics uses batched damped least squares and collision-free paths use RRT-Connect. In the kitchen, each pick–place bundle is planned with PDDLStream (grasp, inverse-kinematics and motion streams; at most 60 s per solve), with placement poses sampled inside the target region's placement area; the mug stored in the cupboard is

picked with a scripted motion, and the box lid is opened by replaying a prerecorded trajectory. In the grill, transfers are executed in stages that are planned as the robot moves, every meat is placed at the fixed pose of the grill placement area, and the grill-lid and plate motions replay prerecorded trajectories. Pre- and post-action checks classify failures such as a closed lid at a placement, a failed grasp or a placement outside the target region.

S3.4 Real-Robot Execution

The physical system uses an xArm manipulator and three Intel RealSense D435 cameras. SAM2.1 provides object segmentation masks, observed objects are associated with predefined placement regions, and these object–region observations update the same memory layer as in simulation. Point clouds from the cameras support grasp generation.

TODO: Real-robot details to be added: the five settings, region definitions, perception and grasping pipeline, and the trials performed.

S4 Prompts and In-Context Examples

The planner receives the same prompt structure in every variant; only the scene’s actions, objects, lids and regions change. All panels are copied from the prompts and trial logs of one evaluated run of our planner (Qwen3-VL-8B-Thinking, ICL condition) unless their title says *summary*.

S4.1 Planning Without In-Context Examples

The zero-shot condition is the shared prompt below: role, the scene’s actions with their preconditions and effects, and the response format, followed by the goal and the scene state. The ICL condition differs only by the examples of Section S4.2, which are appended after this system prompt in grill requests. The panels come from the initial request of a grill trial (G2, seed 0) run with ICL; the appended examples are omitted here, so the text shown is the prompt both conditions share. The kitchen prompt lists only pick, place and open.

INPUT · System prompt (grill; shared by both conditions, ICL examples omitted)

You are the task planner for a robot arm. Given a goal and the current state of a scene, you output the ordered list of actions that
 ↳ achieves the goal.

Actions available in this scene:

pick(o): grasp object o.
Preconditions: the gripper is empty; o is a listed object; o is not in a region closed off by a closed lid.
Effects: the gripper holds o.

place(o, r): put the held object o into region r.
Preconditions: the gripper holds o; r is a listed region; r is not closed off by a closed lid.
Effects: o is in r; the gripper is empty.

open(l): open lid l.
Preconditions: the gripper is empty; l is closed; no object is on the top surface of l.
Effects: l is open; the regions l closes off become reachable and their contents can become visible.

close(l): close lid l.
Preconditions: the gripper is empty; l is open.
Effects: l is closed; the regions l closes off become unreachable and their contents are no longer visible.

Output format:
 Reasoning is allowed. End your answer with a line containing only **FINAL ACTIONS:** followed by one action per line, in execution order, in
 ↳ this form:
FINAL ACTIONS:
 pick(o)
 place(o, r)
 Use only the action names available in this scene and the object, lid and region names given in the state. Write nothing after the last
 ↳ action line. If the goal is already satisfied, write **NO_ACTIONS** as the only line after **FINAL ACTIONS:**.

INPUT · User prompt: initial request (G2)

Goal
 COOK all raw meat using the grill and SERVE all cooked meat on
 ↳ the PLATE in the serving area.

Current state
 Visible **objects** and the region each one is in:
 - plate: dish Rack
 - raw_meat_1: grill_side_area
 Remembered **objects** (not currently visible): last region, steps
 ↳ since last seen:
 - (none)
 Lids:
 - grill_lid: closed (closes off inside_grill)
 Gripper: empty
 Regions: table, grill_side_area, inside_grill, plate_top,
 ↳ serving_area, dish Rack

Information supplied with each request

- The goal g and the observed scene s_t^{vis} : visible objects with their regions, lids and the regions they close off, gripper state and the available regions.
- Remembered objects from m_t , with last region and steps since they were seen (none at the start).
- VLM planners also receive a composite camera image of the scene; LLM planners receive the same text.
- Replanning requests add the completed actions and the remaining plan with action identifiers, the actions scheduled during inference, and the reason for the request; corrective requests replace the output-format paragraph with the block format of Section S4.3.

The planner’s answer may contain reasoning; only the actions after the last **FINAL ACTIONS:** line are parsed and checked before execution.

S4.2 In-Context Examples

In the ICL condition, three examples are appended after the unchanged system prompt of every grill request; kitchen prompts are identical in both conditions. The examples come from unrelated scenes (a laundry dryer and a germination chamber), name no object, lid or region of the evaluated scenes, and state no rule: each shows a successful and an unsuccessful answer and the resulting outcome. They mirror the procedural decisions of the grill without giving grill actions: completing a close–open processing cycle before the final placement, removing an object that is already processed before another cycle, and leaving an unprocessed object in place during a cycle that is already scheduled. The baselines’ ICL rows used Examples 1 and 2 only, rewritten in each baseline’s own input and answer format (Section S5).

Example 1 (laundry): complete the processing cycle before the final placement

Request (summary)

Full-plan request.
Goal
 DRY all wet towels using the dryer and PUT all
 ↳ dry towels in the BASKET in the laundry
 ↳ area.
Current state
 - basket: shelf
 - wet_towel_1: counter
 - dryer_door: closed (closes off inside_dryer)

Good answer

FINAL ACTIONS:
 pick(basket)
 place(basket, laundry_area)
 open(dryer_door)
 pick(wet_towel_1)
 place(wet_towel_1, inside_dryer)
 close(dryer_door)
 open(dryer_door)
 pick(wet_towel_1)
 place(wet_towel_1, basket_top)
 Outcome: the towel came out dry and is in the
 ↳ basket in the laundry area. Goal met.

Bad answer

FINAL ACTIONS:
 open(dryer_door)
 pick(wet_towel_1)
 place(wet_towel_1, inside_dryer)
 pick(wet_towel_1)
 place(wet_towel_1, basket_top)
 Outcome: the towel is still wet, and the basket
 ↳ is still on the shelf. Goal not met.

Example 2 (laundry): remove an already processed object before another cycle

Request (summary)

Corrective request; same goal.
Completed actions
 a1-a3: basket to laundry_area; open(dryer_door)
Remaining plan (not executed yet)
 a4-a5: wet_towel_1 into inside_dryer
 a6: close(dryer_door) a7: open(dryer_door)
 a8-a9: wet_towel_1 to basket_top
Why a new plan is requested
 dry_towel_2 (in inside_dryer): relevant to the
 ↳ goal, not in its goal state

Good answer

FINAL BLOCKS:
BLOCK
objects: dry_towel_2
urgency: urgent
insert: front
reason: dry_towel_2 is already dry.
actions:
 pick(dry_towel_2)
 place(dry_towel_2, basket_top)
END BLOCK
 Outcome: both towels are dry and in the basket.
 ↳ Goal met.

Bad answer

FINAL BLOCKS:
BLOCK
objects: dry_towel_2
urgency: deferred
insert: end
reason: dry_towel_2 can be moved at the end.
actions:
 pick(dry_towel_2)
 place(dry_towel_2, basket_top)
END BLOCK
 Outcome: dry_towel_2 was still inside during a6
 ↳ and a7 and came out scorched. Goal not met.

Example 3 (germination): defer the removal until the scheduled processing is complete

Request (summary)

Corrective request.
Goal
 GERMINATE all seed trays using the germination
 ↳ chamber and PUT all sprouted trays on the
 ↳ NURSERY BENCH.
Completed actions
 a1: open(chamber_door)
Scheduled during planning
 a2-a3: seed_tray_1 into inside_chamber
Remaining plan (not executed yet)
 a4: close(chamber_door) a5: open(chamber_door)
 a6-a7: seed_tray_1 to nursery_bench
Why a new plan is requested
 seed_tray_2 (in inside_chamber): relevant to the
 ↳ goal, not in its goal state; does not lie
 ↳ where the remaining plan places **objects**

Good answer

FINAL BLOCKS:
BLOCK
objects: seed_tray_2
urgency: deferred
insert: after a5
reason: seed_tray_2 has not germinated yet.
actions:
 pick(seed_tray_2)
 place(seed_tray_2, nursery_bench)
END BLOCK
 Outcome: both trays sprouted during a4 and a5
 ↳ and are on the nursery bench. Goal met.

Bad answer

FINAL BLOCKS:
BLOCK
objects: seed_tray_2
urgency: urgent
insert: front
reason: seed_tray_2 must be moved before the
 ↳ door closes.
actions:
 pick(seed_tray_2)
 place(seed_tray_2, nursery_bench)
END BLOCK
 Outcome: seed_tray_2 was taken out before a4 and
 ↳ a5 and never sprouted. Goal not met.

Summary panels condense the request text; the full requests use the format of Section S4.1 (Example 1) and Section S4.3 (Examples 2 and 3), and the answers are reproduced as given in the prompt.

S4.3 Discovery Corrections and Validation Feedback

INPUT · Corrective request: output format (replaces the paragraph of Section S4.1; rest unchanged)

Output format for this request:

Reasoning is allowed. End your answer with a line containing only **FINAL BLOCKS**: followed by one or more blocks. Each block handles some of the **objects** listed
 ↳ under "Why a new plan is requested" and has exactly this form:

BLOCK

objects: <names of the listed **objects** this block handles, comma-separated>

urgency: urgent or deferred

insert: front, or after <action id from the remaining plan>, or end

reason: <one short sentence>

actions:

<one action per line, in execution order>

END BLOCK

Every listed object must be handled by at least one block. An urgent block must use **insert**: front. The actions of a block must end with the gripper empty. If a
 ↳ listed object needs no action, handle it with a block whose actions section is the single line **NO_ACTIONS**; such a block needs no **urgency** or **insert** line. If
 ↳ no listed object needs any action, write **NO_ACTIONS** as the only line after **FINAL BLOCKS**:. Write nothing after the last **END BLOCK**.

Recorded correction (G2, seed 0). Opening the grill revealed one cooked and one raw meat; only a6–a9 are eligible anchors.

INPUT · User prompt (excerpt; goal, state and instructions omitted)

```
## Completed actions
- a1: pick(plate)
- a2: place(plate, serving_area)
- a3: open(grill_lid)

## Scheduled to run before your corrective block is applied (not executed yet)
- a4: pick(raw_meat_1)
- a5: place(raw_meat_1, inside_grill)
- These actions run while you plan. Do not repeat them, and do not use their ids as an
  ↳ insertion point; the Current state does not include their effects yet.

## Remaining plan (not executed yet)
- a6: close(grill_lid)
- a7: open(grill_lid)
- a8: pick(raw_meat_1)
- a9: place(raw_meat_1, plate_top)

## Why a new plan is requested
- After open(grill_lid), these observed objects are not handled by the remaining plan:
  - cooked_meat_1 (in inside_grill): relevant to the goal, not in its goal state; does not
    ↳ lie where the remaining plan places objects
  - raw_meat_2 (in inside_grill): relevant to the goal, not in its goal state; does not
    ↳ lie where the remaining plan places objects
```

OUTPUT · Recorded blocks (parsed fields; reasoning omitted)

```
BLOCK
objects: cooked_meat_1
urgency: urgent
insert: front
reason: it is already cooked but not served on the plate.
actions:
pick(cooked_meat_1)
place(cooked_meat_1, plate_top)
END BLOCK
BLOCK
objects: raw_meat_2
urgency: deferred
insert: after a7
reason: it needs to be cooked before serving, which occurs
  ↳ after the lid is opened.
actions:
pick(raw_meat_2)
place(raw_meat_2, plate_top)
END BLOCK
```

System result: accepted on the first attempt; merged remaining plan $\hat{\pi}$

a10 pick(cooked_meat_1) a11 place(cooked_meat_1, plate_top) a6 close(grill_lid) a7 open(grill_lid)
 a12 pick(raw_meat_2) a13 place(raw_meat_2, plate_top) a8 pick(raw_meat_1) a9 place(raw_meat_1, plate_top)
 During the 142 s inference the robot executed the scheduled bundle a4–a5 (7.5 s). The urgent block enters before the next close; the deferred block follows the cooking cycle a6–a7; the pending actions keep their order and identifiers. The trial succeeded with three FM calls (one initial answer was rejected by the plan check).

Rejected correction and feedback (G1-n1, seed 2). One cooked meat was found; a4–a5 (raw meat into the grill) were scheduled during inference, so the anchor a5 was rejected. Only that anchor of the rejected answer was recorded. The re-query repeats the corrective request with the updated completed actions (a1–a5), adds the feedback line and keeps the remaining plan; the revision was accepted and the trial succeeded.

OUTPUT · Rejected answer (excerpt)

insert: after a5

FEEDBACK · Line added to the re-query

- Your previous blocks were rejected (failure code
 ↳ invalid_corrective_block): block 1 inserts after a5, which is not in
 ↳ the remaining plan

OUTPUT · Revised blocks (parsed fields)

```
BLOCK
objects: cooked_meat_1
urgency: urgent
insert: front
reason: cooked_meat_1 must be served before closing the grill lid to prevent
  ↳ it from being inaccessible.
actions:
pick(cooked_meat_1)
place(cooked_meat_1, plate_top)
END BLOCK
```

S5 Baseline Details

The four external baselines are re-implemented as planning policies within the same pipeline: the same robot, executor and action checks, variants, seeds, planner model and evaluator, with only the planning and replanning logic replaced. None of them uses our memory, *IF* trigger, concurrent execution or corrective insertion. Each receives the visible objects with their regions, the lid and gripper states and the camera image; an object enters its state only once observed and remains known afterwards. The original methods were evaluated with GPT-class models; here all use the selected planner. Planning-call budgets follow each method’s own replanning structure, so planner calls per trial are reported rather than equalized.

The distinctions that matter when reading the comparison are the following. *VLM-TAMP* refines each subgoal before executing it and re-prompts, with its history, only after a subgoal fails, at most twice; objects revealed later therefore enter only through such a failure. *OWL-TAMP* plans once from the initial scene and executes the plan without re-querying the model, so objects hidden at planning time never enter its plan. *Inner Monologue* queries the model after every executed step with success and scene feedback, which makes it the most expensive baseline in calls and time; its running dialogue is embedded in our zero-shot prompt structure in place of the original few-shot task prompts. *EPoG* plans from a goal scene graph built from the goal specification for observed objects, without the original method’s estimation of objects that have not been seen, and calls the model only to resolve simulated motion errors. Its goal graph encodes final object–region relations only, so it cannot represent the cooking procedure; its grill results reflect this representation rather than a planning failure.

ICL rows. The baselines received Examples 1 and 2 of Section S4.2 (the laundry examples) in their own input and answer format, in grill requests only: as a full plan or a replan after a new observation for VLM-TAMP, OWL-TAMP and Inner Monologue, and as error-resolution examples in EPoG’s resolve request. Because EPoG’s model is used only for error resolution and its goal graph has no cooking procedure, the examples cannot change its grill behavior. Kitchen prompts are unchanged, so each baseline’s kitchen results are shared between its zero-shot and ICL rows, as for our method.

S6 Experimental Configuration

Trial protocol. Each variant runs ten trials with seeds 0–9, which set the pose perturbation of Section S2; the suite has 140 trials. Six trials run concurrently against one inference server. A trial may make at most 11 FM calls (the initial request and ten replanning calls) and is stopped after 14,400 s. A trial that ends because of an infrastructure failure (a server error or timeout, or a simulator error) is excluded and rerun, at most twice; every other ending is scored. ICL runs execute only the five grill variants, because kitchen prompts are identical in both conditions; their kitchen results are those of the zero-shot run.

Ablations. Each condition changes one component of the full system. –Memory uses only the current observation. –IF requests a correction for every newly visible object. –WHERE regenerates the remaining plan instead of inserting corrective blocks and also suspends concurrent execution. Fixed front and Fixed end insert

Table S2: Planner inference settings of the selected planner in the reported experiments.

Setting	Value used in the reported experiments
Model	Qwen3-VL-8B-Thinking (BF16 weights), reasoning enabled
Sampling	temperature 1.0, top- <i>p</i> 0.95, top- <i>k</i> 20, min- <i>p</i> 0, presence penalty 0, repetition penalty 1.0
Maximum output length	24,576 tokens per call
Visual input	one composite camera image per request
Inference engine	vLLM 0.30.0, OpenAI-compatible interface, one server per run
Context length	32,768 tokens*
GPU memory utilization	0.90*
Reasoning parsing	Qwen3 reasoning parser; images per request limited to 2*
Request timeout	1,800 s per FM call
Hardware	NVIDIA RTX 5090, 32 GB

*Recorded in the server command of the ICL runs; the zero-shot runs used the same model and sampling, but their server command was not recorded.

every corrective block at the front or the end regardless of the planner’s choice. –WHEN suspends execution during inference. The LLM-planner condition runs the full system with Qwen3-8B with reasoning enabled. Kitchen conditions are zero-shot and grill conditions use the ICL prompt.